ECC 규칙 가이드 – 코딩 규칙이란?

규칙은 "코딩할 때 지켜야 할 약속"입니다. 사람이 글을 쓸 때 맞춤법을 지키는 것처럼, 코드를 쓸 때도 지켜야 할 규칙이 있습니다. ECC는 이 규칙을 자동으로 적용해줍니다.

ECC의 규칙은 크게 두 종류로 나뉩니다:

1. **공통 규칙 (9개)** – 어떤 프로그래밍 언어를 쓰든 반드시 지켜야 할 것들
2. **언어별 규칙 (56개)** – TypeScript, Python, Go, Rust 등 각 언어 고유의 약속들

언어별 규칙은 공통 규칙을 기반으로 하되, 해당 언어에 맞게 구체적으로 확장한 것입니다. 마치 "교통 법규"가 공통이고, "좌측통행(영국)"이나 "우측통행(한국)"이 나라별 규칙인 것과 같습니다.

공통 규칙 (모든 언어에 적용)

1. 코딩 스타일 (coding-style.md)

코드를 깔끔하고 읽기 쉽게 쓰는 방법에 대한 규칙입니다.

- **한번 만든 데이터는 바꾸지 않기 (불변성)** – 원본을 직접 수정하면 다른 곳에서 예상치 못한 문제가 생길 수 있습니다. 대신 복사본을 만들어서 수정합니다. 비유하면, 원본 문서에 직접 낙서하지 말고 복사본에 수정하라는 것입니다
- **파일을 작게 나누기** – 한 파일에 너무 많은 코드를 넣으면 찾기 어렵습니다. 200~400줄이 적당하고, 800줄을 넘기지 않습니다. 서류철 하나에 모든 서류를 넣지 않는 것과 같습니다
- **에러를 반드시 처리하기** – 문제가 생겼을 때 그냥 무시하면 나중에 더 큰 문제가 됩니다. 모든 에러에 대해 "이런 상황이면 이렇게 대처한다"를 미리 정해둡니다
- **외부에서 들어온 데이터는 반드시 검증하기** – 사용자가 입력한 값이나 외부에서 받은 데이터를 그대로 믿으면 안 됩니다. 신분증 확인 없이 건물에 들여보내지 않는 것과 같습니다
- **함수는 50줄 이하, 들여쓰기는 4단계 이하** – 코드를 짧고 단순하게 유지해야 이해하기 쉽습니다
- **숫자나 문자열을 코드에 직접 쓰지 않기** – "매직 넘버"라고 하는데, 의미를 알 수 없는 숫자 대신 이름이 있는 상수를 사용합니다. "7"이라고 쓰는 대신 "일주일_일수 = 7"이라고 쓰는 것입니다

2. 개발 작업 흐름 (development-workflow.md)

새로운 기능을 만들 때 따르는 순서에 대한 규칙입니다.

- **먼저 조사하기** – 새 기능을 만들기 전에, 이미 누군가 만들어 놓은 것이 있는지 먼저 찾아봅니다. 바퀴를 다시 발명하지 않는 것입니다
- **계획을 먼저 세우기** – 코드를 바로 쓰지 말고, 무엇을 어떻게 만들지 설계도를 먼저 그립니다
- **테스트를 먼저 쓰기 (TDD)** – "이 기능이 제대로 동작하면 이런 결과가 나와야 한다"를 먼저 정한 뒤에 코드를 작성합니다. 시험 문제를 먼저 출제하고 답을 쓰는 방식입니다
- **코드 리뷰 받기** – 코드를 쓴 후 자동 검토를 받아서 문제를 조기에 발견합니다

3. Git 작업 흐름 (git-workflow.md)

코드의 변경 이력을 관리하는 방법에 대한 규칙입니다.

- **커밋 메시지 형식 지키기** – 변경 내용을 한눈에 알 수 있도록 정해진 형식으로 메시지를 씁니다. "feat: 로그인 기능 추가", "fix: 비밀번호 오류 수정" 같은 방식입니다
- **PR(풀 리퀘스트) 만들기 전에 전체 변경 내용 확인** – 최근 커밋만 보지 말고, 처음부터 끝까지 무엇이 바뀌었는지 전부 점검합니다
- **테스트 계획 포함하기** – 코드 변경을 올릴 때 "이것을 어떻게 검증할 것인지"도 함께 기록합니다

4. 보안 (security.md)

코드의 안전을 지키는 규칙입니다.

- **비밀번호나 API 키를 코드에 직접 쓰지 않기** – 코드는 여러 사람이 볼 수 있으므로, 비밀 정보는 별도의 안전한 곳에 보관합니다. 비밀번호를 포스트잇에 써서 모니터에 붙이지 않는 것과 같습니다
- **SQL 인젝션 방지** – 사용자 입력을 데이터베이스 명령어에 직접 넣으면 해커가 악용할 수 있습니다. 반드시 안전한 방식으로 처리합니다
- **XSS(크로스사이트 스크립팅) 방지** – 사용자가 입력한 내용이 웹페이지에 그대로 표시되면 악성 코드가 실행될 수 있습니다
- **CSRF 보호** – 다른 사이트가 사용자 모르게 요청을 보내는 공격을 막습니다
- **접속 제한(레이트 리미팅)** – 한 사람이 너무 많은 요청을 보내지 못하게 제한합니다
- **보안 문제 발견 시 즉시 중단** – 보안 문제가 발견되면 다른 모든 작업을 멈추고 보안부터 해결합니다

5. 테스트 (testing.md)

코드가 제대로 동작하는지 확인하는 규칙입니다.

- **테스트 커버리지 80% 이상** – 전체 코드의 80% 이상이 테스트로 검증되어야 합니다
- **3가지 테스트 모두 필수** – 개별 기능 테스트(유닛), 기능 간 연동 테스트(통합), 사용자 흐름 전체 테스트(E2E)를 모두 해야 합니다
- **테스트 먼저 쓰기 (TDD)** – 빨강(실패하는 테스트 작성) -> 초록(테스트를 통과하는 코드 작성) -> 개선(코드 정리) 순서를 반드시 따릅니다

6. 성능 최적화 (performance.md)

시스템을 효율적으로 사용하는 규칙입니다.

- **AI 모델 선택 전략** – 작업의 복잡도에 따라 적절한 AI 모델을 사용합니다. 간단한 일에는 가벼운 모델(Haiku), 복잡한 일에는 강력한 모델(Opus)을 씁니다. 못으로 액자를 걸 때 드릴을 쓸 필요가 없는 것과 같습니다
- **대화 창 관리** – AI와의 대화가 길어지면 기억력이 떨어집니다. 대화 창의 마지막 20%에서는 큰 작업을 시작하지 않습니다
- **깊은 사고 모드 활용** – 복잡한 문제에서는 AI가 더 깊이 생각할 수 있는 모드를 활성화합니다

7. 설계 패턴 (patterns.md)

코드를 구조적으로 잘 짜기 위한 규칙입니다.

- **검증된 뼈대(스켈레톤) 프로젝트 활용** – 새 프로젝트를 시작할 때 검증된 템플릿을 찾아서 기반으로 사용합니다. 집을 지을 때 기본 설계도를 참고하는 것과 같습니다
- **저장소 패턴 (Repository Pattern)** – 데이터를 읽고 쓰는 부분을 일관된 방식으로 관리합니다. 도서관의 대출/반납 시스템처럼 표준화된 인터페이스를 사용합니다
- **통일된 API 응답 형식** – 모든 API 응답에 "성공 여부", "데이터", "에러 메시지"를 항상 같은 형식으로 보냅니다. 택배 송장 양식이 통일되어 있는 것과 같습니다

8. 훅(Hooks) 시스템 (hooks.md)

자동으로 실행되는 검사에 대한 규칙입니다.

- **도구 사용 전 검사 (PreToolUse)** – 파일을 수정하기 전에 자동으로 확인하는 것입니다. 출발 전 안전벨트 확인과 같습니다
- **도구 사용 후 검사 (PostToolUse)** – 파일을 수정한 후 자동으로 정리/포맷하는 것입니다. 요리 후 설거지와 같습니다
- **세션 종료 시 검사 (Stop)** – 작업이 끝날 때 최종 점검을 합니다. 퇴근 전 사무실 불 끄는 것과 같습니다
- **할 일 목록(TodoWrite) 활용** – 여러 단계의 작업을 추적하고, 진행 상황을 실시간으로 보여줍니다

9. 에이전트 조율 (agents.md)

여러 AI 전문가를 활용하는 규칙입니다.

- **전문가별 역할 분담** – 각 에이전트는 전문 분야가 있습니다. 설계는 architect, 테스트는 tdd-guide, 보안은 security-reviewer가 담당합니다
- **복잡한 작업은 자동으로 전문가 호출** – 복잡한 기능 요청이 들어오면 자동으로 planner 에이전트를 사용합니다
- **독립적인 작업은 동시에 실행** – 서로 관련 없는 작업은 여러 에이전트가 동시에 처리합니다. 한 사람이 순서대로 하는 것보다 훨씬 빠릅니다
- **다양한 관점에서 검토** – 복잡한 문제는 사실 확인, 보안 전문가, 일관성 검토 등 여러 관점에서 동시에 분석합니다

TypeScript 규칙 (5개 파일)

TypeScript는 웹 개발에 가장 많이 쓰이는 언어 중 하나입니다. CORTHEX v2도 TypeScript로 만들어져 있습니다.

코딩 스타일 (coding-style.md)

- **타입을 명확하게 정의하기** – 외부에 공개하는 함수는 "이 함수에 무엇을 넣으면 무엇이 나온다"를 명확히 써야 합니다. 택배 상자에 내용물 표시를 하는 것과 같습니다
- **interface vs type 구분** – 물건의 형태를 정의할 때는 interface, 여러 형태를 합치거나 선택할 때는 type을 씁니다
- **any 사용 금지** – any는 "아무거나 됩니다"라는 뜻인데, 이러면 타입 검사가 무의미해집니다. 대신 unknown을 쓰고 확인 후 사용합니다. 신분 미상의 사람을 그냥 통과시키지 않는 것과 같습니다
- **React 컴포넌트의 속성(Props) 타입 정의** – 화면에 보여주는 각 요소가 어떤 데이터를 받는지 명확히 정의합니다
- **스프레드 연산자로 불변 업데이트** – 기존 데이터를 복사하면서 일부만 바꾸는 방식을 사용합니다
- **Zod로 입력값 검증** – 외부에서 들어온 데이터가 올바른 형식인지 자동으로 확인하는 도구를 사용합니다
- **console.log 금지** – 개발 중에 쓰는 디버그 출력을 완성된 코드에 남기지 않습니다

패턴 (patterns.md)

- **통일된 API 응답 형식** – 모든 API 응답에 성공/실패, 데이터, 에러, 페이지 정보를 담습니다
- **커스텀 훅 패턴** – React에서 반복되는 로직을 재사용 가능한 형태로 만듭니다. 자주 쓰는 도구를 공구함에 정리해두는 것입니다
- **저장소 패턴** – 데이터 조회, 생성, 수정, 삭제를 표준화된 인터페이스로 관리합니다

보안 (security.md)

- **비밀 키는 환경 변수로** – API 키를 코드에 직접 쓰지 않고 환경 변수에서 읽습니다
- **보안 리뷰어 에이전트 활용** – 보안 감사를 자동으로 수행합니다

테스트 (testing.md)

- **Playwright로 E2E 테스트** – 사용자가 실제로 사용하는 것처럼 웹 브라우저를 자동으로 조작해서 테스트합니다

훅 (hooks.md)

- **Prettier 자동 포맷** – 파일을 저장하면 자동으로 코드 정리를 합니다. 글을 쓴 후 맞춤법 검사기를 돌리는 것과 같습니다

- **TypeScript 자동 타입 검사** – 파일을 수정하면 자동으로 타입 오류를 확인합니다
- **console.log 경고** – 디버그용 출력이 남아있으면 자동으로 경고합니다
- **세션 종료 시 console.log 전수 검사** – 작업이 끝날 때 수정된 파일 전체에서 console.log를 찾습니다

Python 규칙 (5개 파일)

Python은 데이터 분석, 인공지능, 자동화 등에 많이 쓰이는 언어입니다.

코딩 스타일 (coding-style.md)

- **PEP 8 규약 따르기** – Python 공식 코딩 스타일을 따릅니다. 맞춤법 통일 규칙과 같습니다
- **타입 표시 필수** – 모든 함수에 입력/출력 타입을 명시합니다
- **frozen=True로 불변 데이터 만들기** – 한번 만든 데이터를 수정할 수 없게 잠급니다
- **black으로 자동 포맷, ruff로 오류 검사** – 코드 스타일을 자동으로 맞춰주는 도구를 사용합니다

패턴 (patterns.md)

- **Protocol (덕 타이핑)** – "이런 동작을 할 수 있으면 OK"라는 방식의 인터페이스입니다. 오리처럼 걷고 오리처럼 소리내면 오리다라는 원칙입니다
- **Dataclass로 데이터 전달 객체 만들기** – 데이터의 구조를 깔끔하게 정의합니다
- **컨텍스트 매니저와 제너레이터** – 파일을 열었으면 반드시 닫는 것을 자동으로 처리하고, 큰 데이터를 조금씩 처리합니다

보안 (security.md)

- **환경 변수로 비밀 관리** – API 키를 코드에 쓰지 않고 환경 변수에서 읽습니다
- **bandit로 보안 점검** – 코드의 보안 취약점을 자동으로 찾아주는 도구입니다

테스트 (testing.md)

- **pytest 사용** – Python의 대표적인 테스트 도구를 사용합니다
- **pytest.mark로 테스트 분류** – 유닛 테스트, 통합 테스트 등을 태그로 구분합니다

훅 (hooks.md)

- **black/ruff 자동 포맷** – 파일 저장 시 자동으로 코드를 정리합니다
- **mypy/pyright 타입 검사** – 타입 오류를 자동으로 감지합니다
- **print() 경고** – Python에서도 디버그용 출력을 완성 코드에 남기지 않도록 경고합니다

Go 규칙 (5개 파일)

Go는 서버, 클라우드, 인프라 도구 개발에 많이 쓰이는 언어입니다.

코딩 스타일 (coding-style.md)

- **gofmt, goimports 자동 포맷** – Go 공식 포맷팅 도구를 사용합니다. Go 커뮤니티에서는 스타일 논쟁을 하지 않습니다. 도구가 결정해줍니다
- **인터페이스를 받고 구조체를 반환** – 유연하게 입력을 받되, 구체적인 결과를 돌려주는 설계 원칙입니다
- **에러에 맥락 정보 추가** – 에러가 발생하면 "어디서 무엇을 하다가 실패했다"를 알 수 있도록 정보를 덧붙입니다

패턴 (patterns.md)

- **함수 옵션 패턴** – 설정값이 많을 때 유연하게 옵션을 전달하는 방식입니다
- **인터페이스는 사용하는 곳에서 정의** – 필요한 곳에서 필요한 만큼만 인터페이스를 정의합니다
- **생성자로 의존성 주입** – 필요한 부품을 외부에서 넣어주는 방식으로 테스트가 쉬워집니다

보안 (security.md)

- **gosec로 보안 점검** – Go 전용 보안 취약점 검사 도구입니다
- **context.Context로 타임아웃 설정** – 작업이 너무 오래 걸리면 자동으로 중단하는 안전장치입니다

테스트 (testing.md)

- **테이블 주도 테스트** – 여러 입력/출력 조합을 표 형태로 정리해서 한번에 테스트합니다
- **-race 플래그 필수** – 여러 작업이 동시에 실행될 때 충돌하는 문제를 감지합니다

훅 (hooks.md)

- **gofmt/goimports 자동 포맷** – Go 파일 수정 후 자동 정리
- **go vet 정적 분석** – 흔한 실수를 자동으로 찾아줍니다
- **staticcheck 확장 검사** – 더 정밀한 코드 분석을 실행합니다

Rust 규칙 (5개 파일)

Rust는 안전성과 성능을 모두 잡은 시스템 프로그래밍 언어입니다.

코딩 스타일 (coding-style.md)

- **rustfmt, clippy 필수** – 공식 포맷팅 도구와 린트 도구를 반드시 사용합니다
- **기본적으로 불변** – Rust는 변수가 기본적으로 변경 불가합니다. 변경이 필요할 때만 mut을 붙입니다
- **소유권과 빌림** – Rust만의 독특한 규칙입니다. 데이터의 주인이 누구인지 명확히 하고, 빌려 쓸 때는 규칙을 따릅니다. 도서관 책처럼 빌리고 반납하는 시스템입니다
- **Result와 ?로 에러 전파** – unwrap()(강제 풀기)을 쓰면 프로그램이 갑자기 멈출 수 있으므로, 에러를 안전하게 전달합니다
- **반복자(Iterator) 선호** – 데이터를 하나씩 처리할 때 깔끔한 체인 방식을 사용합니다
- **도메인별 모듈 구성** – 기능별로 폴더를 나누어 관리합니다

패턴 (patterns.md)

- **트레이트(Trait)로 저장소 패턴** – 데이터 접근 방식을 추상화합니다
- **Newtype 패턴** – 같은 숫자 타입이라도 "사용자 ID"와 "주문 ID"를 구분해서 실수로 섞이지 않게 합니다
- **열거형(Enum) 상태 기계** – 프로그램의 상태를 열거형으로 표현해서, 불가능한 상태 자체를 만들 수 없게 합니다
- **빌더 패턴** – 설정이 많은 객체를 단계별로 조립합니다
- **봉인된 트레이트 (Sealed Trait)** – 외부에서 마음대로 확장하지 못하게 제한합니다

보안 (security.md)

- **unsafe 최소화** – Rust의 안전 보장을 해제하는 코드는 최소한으로 쓰고, 반드시 이유를 설명합니다
- **cargo audit로 취약점 검사** – 사용하는 외부 라이브러리에 알려진 보안 문제가 없는지 확인합니다
- **SQL 인젝션 방지** – 반드시 매개변수화된 쿼리를 사용합니다
- **에러 메시지에 내부 정보 노출 금지** – 사용자에게는 일반적인 메시지만 보여주고, 상세 내용은 서버 로그에만 기록합니다

테스트 (testing.md)

- **#[test]와 #[cfg(test)] 모듈** – 테스트 코드를 해당 기능 바로 옆에 둡니다
- **rstest로 매개변수 테스트** – 여러 입력값을 한번에 테스트합니다
- **mockall로 모의 객체** – 테스트할 때 실제 외부 서비스 대신 가짜를 사용합니다
- **cargo-llvm-cov로 커버리지 80% 이상** – 테스트가 코드의 80% 이상을 검증하는지 확인합니다

훅 (hooks.md)

- **cargo fmt 자동 포맷** – 파일 수정 후 자동 정리
- **cargo clippy 린트 검사** – 코드의 잠재적 문제를 자동 감지
- **cargo check 컴파일 확인** – 코드가 올바르게 컴파일되는지 빠르게 확인

Java 규칙 (5개 파일)

Java는 기업 시스템, 안드로이드 앱 등에 널리 쓰이는 언어입니다.

코딩 스타일 (coding-style.md)

- **google-java-format으로 자동 포맷** – 구글 스타일 가이드를 따릅니다
- **record로 불변 값 타입** – 데이터를 담는 간결한 구조를 사용합니다 (Java 16+)
- **sealed class로 닫힌 타입 계층** – "이 타입의 하위 타입은 이것들만 가능"이라고 제한합니다 (Java 17+)
- **패턴 매칭** – 타입 확인과 변환을 한번에 합니다 (Java 16+)
- **Optional 사용법** – 결과가 없을 수 있는 경우에 사용하되, 함수 매개변수로는 쓰지 않습니다
- **스트림(Stream)** – 데이터 변환은 3~4단계 이내로 간결하게 합니다

패턴 (patterns.md)

- **저장소 패턴과 서비스 계층** – 데이터 접근과 비즈니스 로직을 분리합니다
- **생성자 주입 필수** – 필요한 부품을 외부에서 주입합니다. 필드에 직접 주입하면 테스트가 어려워집니다
- **DTO 매핑** – 외부와 데이터를 주고받을 때 전용 객체를 사용합니다
- **빌더 패턴** – 옵션이 많은 객체를 단계별로 만듭니다
- **Sealed 타입** – 결제 성공/실패처럼 정해진 결과만 표현합니다

보안 (security.md)

- **PreparedStatement 필수** – SQL 인젝션을 방지하기 위해 매개변수화된 쿼리만 사용합니다
- **Bean Validation으로 입력 검증** – 데이터가 유효한지 자동으로 확인합니다
- **커스텀 인증 금지** – 검증된 라이브러리를 사용합니다. 직접 만든 보안 코드는 위험합니다
- **bcrypt/Argon2로 비밀번호 저장** – MD5나 SHA1은 너무 쉽게 뚫립니다
- **OWASP Dependency-Check** – 사용 중인 라이브러리의 보안 취약점을 검사합니다

테스트 (testing.md)

- **JUnit 5** + **AssertJ** + **Mockito** – Java의 표준 테스트 도구 조합입니다
- **Testcontainers** – 테스트 시 실제 데이터베이스를 일시적으로 실행합니다
- **매개변수 테스트** – 여러 입력값 조합을 CSV로 정리해서 한번에 테스트합니다
- **JaCoCo로 커버리지 80% 이상** – 테스트 범위를 확인합니다

훅 (hooks.md)

- **google-java-format 자동 포맷** – 파일 수정 후 자동 정리

- **checkstyle 스타일 검사** – 코딩 규칙 준수 여부를 확인합니다
- **mvnw compile / gradlew compileJava** – 코드가 올바르게 컴파일되는지 확인합니다

Kotlin 규칙 (5개 파일)

Kotlin은 안드로이드 앱과 멀티플랫폼 개발에 쓰이는 현대적인 언어입니다.

코딩 스타일 (coding-style.md)

- **val 우선 사용** – 변경 불가능한 변수를 기본으로 쓰고, 꼭 필요할 때만 var을 씁니다
- **!! 사용 금지** – 강제 널 해제는 프로그램을 갑자기 멈추게 할 수 있습니다. 안전한 방법(?., ?:)을 사용합니다
- **sealed class/interface** – 가능한 상태를 한정합니다. 로딩 중/성공/에러처럼 정해진 상태만 허용합니다
- **확장 함수(Extension Function)** – 기존 타입에 새 기능을 추가하되, 찾기 쉬운 곳에 모아둡니다
- **스코프 함수 적절히 사용** – let, run, apply, also를 상황에 맞게 씁니다. 2단계 이상 중첩하지 않습니다
- **CancellationException 재발생** – 코루틴 취소 예외는 절대로 잡지 않고 그대로 전달합니다

패턴 (patterns.md)

- **ViewModel 패턴** – 화면의 상태를 하나의 객체로 관리하고, 한 방향으로만 데이터가 흐릅니다
- **UseCase 패턴** – 하나의 비즈니스 동작을 하나의 클래스에 담습니다
- **expect/actual (KMP)** – 같은 코드를 안드로이드와 iOS에서 쓸 때, 플랫폼마다 다른 부분만 따로 구현합니다
- **코루틴 패턴** – 비동기 작업을 관리하는 방법을 표준화합니다
- **DSL 빌더 패턴** – 설정을 마치 문장처럼 읽기 쉽게 작성합니다

보안 (security.md)

- **BuildConfig로 비밀 관리** – API 키를 빌드 시점에 안전하게 주입합니다
- **EncryptedSharedPreferences / Keychain** – 민감한 데이터를 암호화해서 저장합니다
- **HTTPS 필수, 일반 HTTP 차단** – 네트워크 통신은 항상 암호화합니다
- **매개변수화 쿼리** – Room/SQLDelight에서 사용자 입력을 SQL에 안전하게 넣습니다
- **ProGuard/R8 규칙** – 앱을 배포할 때 코드를 난독화하되, 직렬화 모델은 보호합니다
- **WebView 보안** – 웹뷰에서 JavaScript를 필요할 때만 활성화하고, 민감한 데이터 접근을 차단합니다

테스트 (testing.md)

- **Turbine으로 Flow 테스트** – 실시간 데이터 흐름을 테스트합니다
- **Fake을 Mock보다 선호** – 직접 만든 가짜 객체가 모킹 프레임워크보다 더 명확합니다
- **runTest로 코루틴 테스트** – 비동기 코드를 동기적으로 테스트합니다
- **Room/SQLDelight 인메모리 테스트** – 임시 데이터베이스로 빠르게 테스트합니다

훅 (hooks.md)

- **ktfmt/ktlint 자동 포맷** – 파일 수정 후 자동 정리
- **detekt 정적 분석** – 코드 품질 문제를 자동 감지
- **gradlew build 컴파일 확인** – 변경 후 빌드가 되는지 확인

Swift 규칙 (5개 파일)

Swift는 Apple(iPhone, Mac 등) 앱 개발에 쓰이는 언어입니다.

코딩 스타일 (coding-style.md)

- **let 우선 사용** – 변수는 기본적으로 변경 불가, 꼭 필요할 때만 var을 씁니다
- **struct 우선 사용** – 값을 복사하는 구조체를 기본으로 쓰고, 참조가 필요할 때만 class를 씁니다
- **Apple API 디자인 가이드라인 따르기** – "사용하는 곳에서 명확하게" 이름을 짓습니다
- **typed throws (Swift 6+)** – 어떤 종류의 에러가 발생할 수 있는지 타입으로 명시합니다
- **Actor로 동시성 관리** – 여러 작업이 동시에 데이터를 수정할 때 충돌을 방지합니다

패턴 (patterns.md)

- **프로토콜 지향 설계** – 작고 집중된 프로토콜(인터페이스)을 만들고, 기본 구현을 제공합니다
- **값 타입과 열거형** – 상태를 열거형으로 모델링합니다 (idle, loading, loaded, failed)
- **Actor 패턴** – 공유 데이터를 안전하게 관리하는 캐시 등에 사용합니다
- **기본 매개변수로 의존성 주입** – 운영 환경에서는 기본값을 쓰고, 테스트에서는 가짜를 주입합니다

보안 (security.md)

- **Keychain Services 사용** – 비밀번호와 토큰은 Keychain에 저장합니다. UserDefaults는 안전하지 않습니다
- **App Transport Security (ATS)** – 안전한 통신만 허용하는 Apple의 기본 보안을 유지합니다
- **입력 검증** – Deep link, 클립보드 등 외부 입력을 반드시 검증합니다

테스트 (testing.md)

- **Swift Testing 프레임워크** – @Test와 #expect를 사용하는 최신 테스트 방식입니다
- **매개변수 테스트** – 여러 입력값을 한번에 테스트합니다
- **테스트 격리** – 각 테스트는 독립적으로 실행됩니다. 테스트 간에 상태를 공유하지 않습니다

훅 (hooks.md)

- **SwiftFormat 자동 포맷** – 파일 수정 후 자동 정리
- **SwiftLint 린트 검사** – 스타일 규칙 준수 확인
- **swift build 컴파일 확인** – 변경 후 빌드가 되는지 확인
- **print() 경고** – 운영 코드에서는 os.Logger를 사용해야 합니다

C++ 규칙 (5개 파일)

C++는 게임, 운영체제, 임베디드 시스템 등 고성능이 필요한 곳에 쓰이는 언어입니다.

코딩 스타일 (coding-style.md)

- **현대 C++ (C++17/20/23) 기능 사용** – 오래된 C 스타일 대신 안전하고 현대적인 문법을 씁니다
- **RAII (자원 관리)** – 메모리를 수동으로 할당/해제하지 않고, 자동으로 관리되는 스마트 포인터를 사용합니다. 자동 잠금 문처럼, 열린 것은 자동으로 닫힙니다
- **clang-format 필수** – 코드 스타일을 자동으로 통일합니다

패턴 (patterns.md)

- **RAII 패턴** – 객체가 생성될 때 자원을 얻고, 소멸될 때 자동으로 반환합니다
- **Rule of Five/Zero** – 특별한 관리가 필요 없으면 아무것도 정의하지 않고(Zero), 하나라도 정의하면 다섯 개를 모두 정의합니다(Five)
- **값 의미론** – 작은 데이터는 복사, 큰 데이터는 참조, 반환은 값으로 합니다
- **std::optional, std::expected** – 값이 없을 수 있거나 에러가 예상되는 경우를 안전하게 표현합니다

보안 (security.md)

- **스마트 포인터 필수, raw new/delete 금지** – 메모리 누수와 이중 해제를 방지합니다
- **C 스타일 배열 금지** – 경계를 넘는 접근으로 인한 취약점을 막기 위해 std::vector 등을 사용합니다
- **strcpy, sprintf 금지** – 버퍼 오버플로우 위험이 있는 함수를 사용하지 않습니다
- **변수는 반드시 초기화** – 초기화하지 않은 변수는 예측 불가능한 동작을 합니다
- **AddressSanitizer, UndefinedBehaviorSanitizer** – 메모리 오류와 정의되지 않은 동작을 감지하는 도구입니다

테스트 (testing.md)

- **GoogleTest (gtest/gmock)** – C++의 표준적인 테스트 프레임워크입니다
- **Sanitizer와 함께 테스트** – CI에서 항상 메모리 안전 검사를 함께 실행합니다
- **lcov로 커버리지** – 테스트가 코드를 얼마나 검증하는지 확인합니다

훅 (hooks.md)

- **clang-format 포맷 검사** – 코드 스타일 확인
- **clang-tidy 정적 분석** – 잠재적 문제 감지
- **cmake build** + **ctest** – 빌드 및 테스트 자동 실행

C# 규칙 (5개 파일)

C#은 Windows 앱, 게임(Unity), 웹 서비스 등에 널리 쓰이는 언어입니다.

코딩 스타일 (coding-style.md)

- **record로 불변 모델** – 데이터를 담는 변경 불가능한 타입을 간결하게 만듭니다
- **init 세터, 불변 컬렉션** – 공유 상태를 변경할 수 없게 합니다
- **async/await 사용, .Result/.Wait() 금지** – 비동기 코드를 안전하게 처리합니다. 블로킹 호출은 교착 상태를 일으킬 수 있습니다
- **CancellationToken 전달** – 비동기 작업을 중간에 취소할 수 있게 합니다
- **dotnet format으로 자동 포맷** – 코드 스타일을 자동으로 맞춥니다

패턴 (patterns.md)

- **저장소 패턴과 API 응답 형식** – 데이터 접근과 응답을 표준화합니다
- **Options 패턴** – 설정값을 강타입으로 관리합니다. 문자열로 접근하면 오타가 생길 수 있습니다
- **의존성 주입** – 인터페이스에 의존하고, 생명주기(singleton, scoped, transient)를 의도적으로 설정합니다

보안 (security.md)

- **User Secrets로 로컬 비밀 관리** – 개발 환경에서 비밀을 안전하게 관리합니다
- **매개변수화 쿼리 필수** – ADO.NET, Dapper, EF Core에서 SQL 인젝션을 방지합니다
- **FluentValidation으로 입력 검증** – 복잡한 유효성 검사를 깔끔하게 작성합니다
- **스택 트레이스 노출 금지** – API 응답에 내부 정보를 포함하지 않습니다

테스트 (testing.md)

- **xUnit** + **FluentAssertions** + **Moq/NSubstitute** – C#의 표준 테스트 도구 조합입니다
- **WebApplicationFactory** – ASP.NET Core API를 실제 HTTP 요청으로 테스트합니다
- **Testcontainers** – 실제 인프라를 임시로 실행해서 테스트합니다

훅 (hooks.md)

- **dotnet format 자동 포맷** – 파일 수정 후 자동 정리
- **dotnet build 컴파일 확인** – 변경 후 빌드가 되는지 확인
- **dotnet test 테스트 실행** – 동작 변경 시 관련 테스트를 재실행합니다
- **appsettings 경고** – 설정 파일에 비밀이 포함되지 않도록 경고합니다

PHP 규칙 (5개 파일)

PHP는 워드프레스를 비롯한 웹 서비스에 널리 쓰이는 언어입니다.

코딩 스타일 (coding-style.md)

- **PSR-12 규약 따르기** – PHP 공식 코딩 스타일을 따릅니다
- **strict_types 선언** – 타입을 엄격하게 검사합니다
- **readonly 속성으로 불변 DTO** – 데이터 전달 객체를 변경 불가능하게 만듭니다
- **PHP-CS-Fixer 또는 Laravel Pint** – 코드 포맷을 자동으로 맞춥니다
- **PHPStan 또는 Psalm** – 타입 오류를 자동으로 찾습니다

패턴 (patterns.md)

- **얇은 컨트롤러, 명확한 서비스** – 컨트롤러는 요청 처리만, 비즈니스 로직은 서비스에 둡니다
- **DTO와 값 객체** – 연관 배열 대신 명확한 구조를 사용합니다
- **의존성 주입** – 인터페이스에 의존하고, 생성자로 주입합니다
- **외부 SDK 래핑** – 외부 라이브러리를 직접 쓰지 않고 어댑터로 감쌉니다

보안 (security.md)

- **FormRequest 등으로 입력 검증** – 요청 데이터를 프레임워크 경계에서 검증합니다
- **기본 이스케이프 출력** – 템플릿에서 HTML을 기본적으로 이스케이프합니다
- **prepared statement 필수** – PDO, Doctrine, Eloquent에서 SQL 인젝션을 방지합니다
- **mass-assignment 제한** – 대량 할당 시 쓰기 가능한 필드를 명시적으로 지정합니다
- **password_hash/verify** – 비밀번호를 안전하게 저장합니다
- **composer audit** – 사용 중인 패키지의 보안 취약점을 검사합니다
- **세션 ID 재생성** – 인증 후 세션을 갱신해서 세션 고정 공격을 방지합니다

테스트 (testing.md)

- **PHPUnit 또는 Pest** – PHP의 대표적인 테스트 프레임워크입니다
- **pcov/Xdebug로 커버리지** – 테스트 범위를 확인합니다
- **빠른 유닛 테스트와 통합 테스트 분리** – 빠른 테스트와 느린 테스트를 분리합니다
- **Inertia.js 사용 시 assertInertia** – SPA와 서버의 연동을 테스트합니다

훅 (hooks.md)

- **Pint/PHP-CS-Fixer 자동 포맷** – 파일 수정 후 자동 정리
- **PHPStan/Psalm 정적 분석** – 타입 오류 자동 감지
- **var_dump, dd, die 경고** – 디버그용 코드가 남아있으면 경고합니다
- **raw SQL, CSRF 비활성화 경고** – 보안에 위험한 변경이 감지되면 경고합니다

Perl 규칙 (5개 파일)

Perl은 텍스트 처리, 시스템 관리, 웹 개발 등에 쓰이는 언어입니다.

코딩 스타일 (coding-style.md)

- **use v5.36 필수** – strict, warnings가 자동 활성화되어 실수를 잡아줍니다
- **서브루틴 시그니처 사용** – @_를 직접 풀지 않고 현대적인 방식을 씁니다
- **Moo + is => 'ro'로 불변 객체** – 읽기 전용 속성만 가진 객체를 만듭니다
- **perltidy로 자동 포맷** – 4칸 들여쓰기, 100자 줄 길이를 자동 적용합니다
- **perlcritic으로 린트** – Perl 모범 사례를 자동으로 검사합니다

패턴 (patterns.md)

- **DBI/DBIx::Class 저장소 패턴** – 데이터베이스 접근을 인터페이스로 관리합니다
- **Moo + Types::Standard으로 DTO** – 데이터 구조를 타입과 함께 정의합니다
- **autodie + Path::Tiny** – 파일 작업에서 에러를 자동으로 처리하고, 경로를 간단하게 다룹니다
- **@EXPORT_OK만 사용** – 외부에 공개하는 함수를 명시적으로 지정합니다
- **cpanfile + carton** – 의존성을 정확한 버전으로 관리합니다

보안 (security.md)

- **Taint 모드 (-T)** – 외부 입력을 자동으로 "오염"으로 표시해서 검증 없이 사용하지 못하게 합니다
- **허용 목록 기반 검증** – "이것만 허용"하는 방식으로 입력을 검증합니다
- **3인자 open 필수** – 2인자 open은 보안 취약점이 있습니다
- **목록 형식 system() 필수** – 문자열 형식은 셸 인젝션 위험이 있습니다
- **DBI 플레이스홀더 필수** – SQL 인젝션을 방지합니다
- **perlcritic security 테마** – 보안 관련 모범 사례를 자동 검사합니다

테스트 (testing.md)

- **Test2::V0 사용** – Perl의 현대적인 테스트 프레임워크입니다
- **prove -l로 실행** – lib/를 경로에 포함시켜 테스트합니다
- **Devel::Cover로 커버리지 80% 이상** – 테스트 범위를 확인합니다
- **done_testing 필수** – 테스트 파일 끝에 반드시 추가합니다

훅 (hooks.md)

- **perltidy 자동 포맷** – 파일 수정 후 자동 정리
- **perlcritic 린트 검사** – 모범 사례 준수 확인
- **print 경고** – 모듈(.pm) 파일에서는 say나 로깅 모듈을 사용해야 합니다

규칙 우선순위

언어별 규칙과 공통 규칙이 충돌하면, **언어별 규칙이 우선**합니다.

예를 들어, 공통 규칙에서는 "데이터를 변경하지 마라(불변성)"라고 하지만, Go 언어에서는 포인터 리시버로 구조체를 직접 변경하는 것이 관례이므로 Go의 방식을 따릅니다.

이것은 "전국 법률"보다 "지역 조례"가 특정 상황에서 더 구체적인 것과 같습니다.

총 65개 규칙 파일 요약

분류	파일 수	핵심 주제
공통	9개	코딩 스타일, 개발 흐름, Git, 보안, 테스트, 성능, 패턴, 훅, 에이전트
TypeScript	5개	타입 안전, Zod 검증, React 패턴, Playwright E2E, Prettier 자동 포맷
Python	5개	PEP 8, pytest, bandit 보안, black 포맷, 타입 힌트
Go	5개	gofmt, 테이블 테스트, gosec, 에러 래핑, race 감지
Rust	5개	소유권/빌림, cargo clippy, thiserror/anyhow, unsafe 최소화, 매개변수 테스트
Java	5개	record/sealed, JUnit 5, PreparedStatement, google-java-format, OWASP
Kotlin	5개	val/sealed, Turbine, EncryptedSharedPreferences, ktlint, KMP expect/actual
Swift	5개	let/struct, Swift Testing, Keychain, SwiftFormat, Actor 동시성
C++	5개	RAII/스마트 포인터, GoogleTest, Sanitizer, clang-format/tidy, std::optional
C#	5개	record/init, xUnit, FluentAssertions, dotnet format, Options 패턴
PHP	5개	PSR-12, PHPUnit/Pest, prepared statement, Pint, strict_types
Perl	5개	v5.36, Test2::V0, Taint 모드, perltidy/critic, Moo 불변